

EXERCICE5

Compte rendu Exercice 5 : analyse Infrastructure as Code avec Checkov

Méthode utilisée

J'ai d'abord cloné le dépôt cible :

```
git clone https://github.com/bridgecrewio/terragoat ./target
```

Le dépôt contient plusieurs environnements cloud. Pour rester cohérent avec l'exercice, j'ai ciblé uniquement la partie AWS :

```
target/terraform/aws
```

J'ai ensuite lancé Checkov avec Docker Compose :

```
docker compose run --rm checkov
```

La commande exécutée dans le conteneur est :

```
checkov -d /tf/terraform/aws -o json > /reports/checkov-report.json || true  
checkov -d /tf/terraform/aws --compact || true
```

Le `|| true` est volontaire. Checkov retourne un code de sortie non nul lorsqu'il détecte des contrôles en échec.

Résultat du scan

Le scan Terraform donne le résultat suivant :

```
Passed checks: 115  
Failed checks: 215  
Skipped checks: 0
```

Checkov analyse aussi un Dockerfile et les secrets présents dans le dépôt :

```
dockerfile scan results:  
Passed checks: 2  
Failed checks: 2  
Skipped checks: 0  
  
secrets scan results:
```

```
Passed checks: 0
Failed checks: 4
Skipped checks: 0
```

Au total, le bilan est donc :

```
passed: 117
failed: 221
ressources concernées: 47
```

Les contrôles qui reviennent le plus souvent concernent surtout les bases RDS/Aurora et les buckets S3 :

```
9x chiffrement Aurora absent
9x protection contre la suppression RDS absente
9x chiffrement KMS des clusters RDS absent
9x authentification IAM RDS désactivée
9x audit/logs RDS ou Aurora non activés
9x plan de sauvegarde AWS Backup absent pour les clusters RDS
6x bucket S3 sans blocage d'accès public
```

Analyse des risques principaux

Les résultats montrent plusieurs familles de mauvaises configurations.

La première concerne les secrets. Checkov détecte des clés AWS écrites en dur dans le code Terraform. C'est un risque majeur, car une clé exposée peut permettre à un attaquant d'agir directement sur le compte cloud.

La deuxième concerne l'exposition réseau. Le port SSH est ouvert à `0.0.0.0/0`, ce qui signifie qu'il est accessible depuis Internet. Ce type de configuration augmente fortement le risque de tentatives de brute force ou d'exploitation.

La troisième concerne les services managés AWS. Certaines bases RDS ne sont pas correctement protégées : chiffrement absent, sauvegardes insuffisantes, protection contre la suppression non activée ou accessibilité publique. Ces erreurs peuvent entraîner une fuite, une altération ou une perte de données.

Enfin, les buckets S3 ne disposent pas toujours d'un blocage d'accès public. C'est une mauvaise pratique fréquente qui peut exposer des données sensibles si une politique d'accès est mal configurée.

Réponses aux questions

Combien de contrôles échouent ?

221 contrôles échouent au total, contre 117 contrôles passés. Les échecs concernent 47 ressources différentes.

Quelles sont les erreurs les plus critiques ?

Les erreurs les plus critiques sont les clés AWS écrites en dur, le port SSH ouvert à Internet, une base RDS accessible publiquement, un endpoint EKS public et des buckets S3 sans blocage d'accès public.

Quels risques représentent-elles ?

Les clés AWS exposées peuvent permettre une compromission directe du compte cloud. Le SSH ouvert augmente la surface d'attaque. Une base RDS publique ou un bucket S3 mal protégé peut provoquer une fuite de données. Un endpoint EKS public expose l'API Kubernetes et doit être strictement contrôlé.

Comment les corriger ?

Les clés AWS doivent être supprimées du code et remplacées par des rôles IAM, des variables sécurisées ou un coffre-fort de secrets. Les security groups doivent être restreints aux adresses IP nécessaires. Les bases RDS doivent être placées dans des sous-réseaux privés, chiffrées, sauvegardées et protégées contre la suppression. Les buckets S3 doivent avoir le blocage d'accès public activé, ainsi que le chiffrement et la journalisation.

Priorisation proposée

La première priorité serait de traiter les secrets exposés, car ils peuvent permettre une compromission immédiate. Ensuite, je corrigerais les ressources publiques, notamment SSH, RDS, EKS et S3. Enfin, je traiterais les contrôles de durcissement comme les logs, les sauvegardes, le chiffrement KMS et les politiques IAM trop permissives.

Dans une chaîne DevSecOps, Checkov devrait être intégré à la CI/CD afin de bloquer automatiquement les configurations dangereuses avant leur déploiement.